

Universidad de La Laguna
Sistemas Ciberfísicos – Curso 25/26

Documento técnico final
Sistema ciberfísico: 3 en raya con Ned-2

Pablo Navarro Domínguez

12 de mayo de 2026

Índice

1. Resumen	3
2. Alcance del documento final	3
3. Descripción técnica de la solución final	4
3.1. Arquitectura general del sistema	4
3.2. Subsistemas software desarrollados	5
3.3. Características principales y decisiones de diseño por subsistema	6
3.3.1. PC central (Simulink + control de partida)	6
3.3.2. Subsistema robot (control de movimiento)	6
3.3.3. Sistema de visión	7
3.3.4. Comunicación TCP/IP	7
3.3.5. IA y decisión automática	8
3.4. Observaciones de consistencia detectadas en el código actual	9
4. Validación pormenorizada de requisitos	10
4.1. Procedimiento general de validación	10
4.2. Estado observado en código actual (auditoría de implementación)	10
4.3. Requisitos pendientes por verificar (tabla preparada)	11
4.4. Plantilla de ficha de verificación individual	12
5. Guía rápida para la puesta en marcha	13
5.1. Prerrequisitos	13
5.2. Secuencia de arranque recomendada	13
5.3. Comprobaciones de arranque (smoke test)	14
5.4. Checklist de configuración inicial	14
5.5. Incidencias frecuentes y acción rápida	14
6. Anexos de evidencias	14
6.1. Índice de evidencias (plantilla)	14
6.2. Anexo de trazas de comunicación (pendiente)	15
6.3. Anexo fotográfico/video (pendiente)	15
7. Cierre (plantilla)	15

1. Resumen

Este documento técnico final consolida el desarrollo del sistema ciberfísico de 3 en raya con Ned-2, integrando los avances recogidos en los dos documentos técnicos previos y alineándolos con el enunciado oficial del proyecto. El objetivo funcional del sistema es ejecutar una partida física completa frente a un rival humano, combinando supervisión en tiempo real, manipulación segura, detección visual del tablero y comunicación remota entre nodos de control.

La solución no se ha limitado a una descripción abstracta del sistema, sino que se apoya en componentes concretos ya desarrollados: ComRobot.py como núcleo de control de partida en el robot, Ned2_ULL.py como conjunto de utilidades de configuración y persistencia de poses, y los bloques SimulinkEmisor.m y SimulinkReceptor.m como interfaz entre el PC central y el protocolo TCP/IP.

La implementación actual se estructura en dos capas de decisión y ejecución: el PC central, responsable del control supervisado mediante máquina de estados en Simulink, y el PC del robot, donde se ejecuta la lógica Python de movimiento, visión, inteligencia artificial y mensajería. Esta separación ha permitido mantener trazabilidad de eventos, reducir acoplamiento entre subsistemas y facilitar la verificación incremental por hitos.

En el estado presente del proyecto se dispone de funcionamiento operativo de comunicación y ejecución de turnos en modo manual y automático, con protocolo TCP/IP activo, detección visual integrada y secuencias de movimiento pick-and-place estables. No obstante, el cierre automático de partida y la emisión sistemática de mensaje FIN en el flujo principal del robot siguen en consolidación de integración. El cierre final del documento se completará con las evidencias experimentales restantes y con los resultados de la versión definitiva de código.

2. Alcance del documento final

El alcance de este informe incluye la descripción técnica de la solución desarrollada en el PC central, el PC de control del robot, el subsistema de visión y la capa de comunicaciones. También cubre las decisiones de diseño adoptadas para satisfacer las restricciones del enunciado, en particular las asociadas a seguridad cinemática, sincronización de turnos y capacidad de supervisión remota.

Además, se incorporan detalles de hardware y software que ya aparecían en los dos documentos previos: la configuración de visión del Ned-2, la posición home como estado seguro de inicio, las limitaciones temporales de recogida y transporte, el uso de mensajes de texto terminados en salto de línea y el papel de la IA en el modo automático.

Desde el punto de vista de verificación, el documento organiza el estado de cumplimiento de requisitos, indicando qué aspectos ya fueron validados en entregables anteriores y qué aspectos permanecen pendientes para la campaña final de pruebas en campo. Se mantiene, además, una estructura de evidencias compatible con la trazabilidad exigida por el cliente.

Quedan fuera del alcance de esta revisión la redacción de conclusiones definitivas de validación y la consolidación de resultados de la ultima iteración de software, que dependen de pruebas adicionales sobre el sistema global.

Hito	Objetivo de proyecto	Cobertura documental actual	Estado
H1	Aprobación del documento de requisitos	Referencia de requisitos y matriz de validación	Completado
H2	Verificación del PC central	Arquitectura de control y protocolo Simulink	Avanzado
H3	Verificación del robot manipulador	Control de movimiento, visión y seguridad	Avanzado
H4	Validación del sistema ciberfísico	Procedimientos y evidencias de cierre	En curso

Tabla 1: Cobertura del documento respecto a los hitos del proyecto.

3. Descripción técnica de la solución final

3.1. Arquitectura general del sistema

La arquitectura final implementa un esquema distribuido de supervisión y ejecución. El PC central actúa como nodo de control de alto nivel y concentra la interfaz de usuario, la máquina de estados y la coordinación de la partida. El PC del robot ejecuta la capa de campo: control de movimiento, percepción visual, cálculo de jugada en modo automático y notificación de eventos al sistema central.

El ciclo operativo comienza con la selección de modo y la verificación de precondiciones de seguridad (home + Start), continúa con la alternancia de turnos y finaliza cuando se detecta una condición de victoria, derrota o empate. Esta estructura se diseñó para satisfacer las exigencias del enunciado sobre supervisión continua del estado y control remoto del robot.

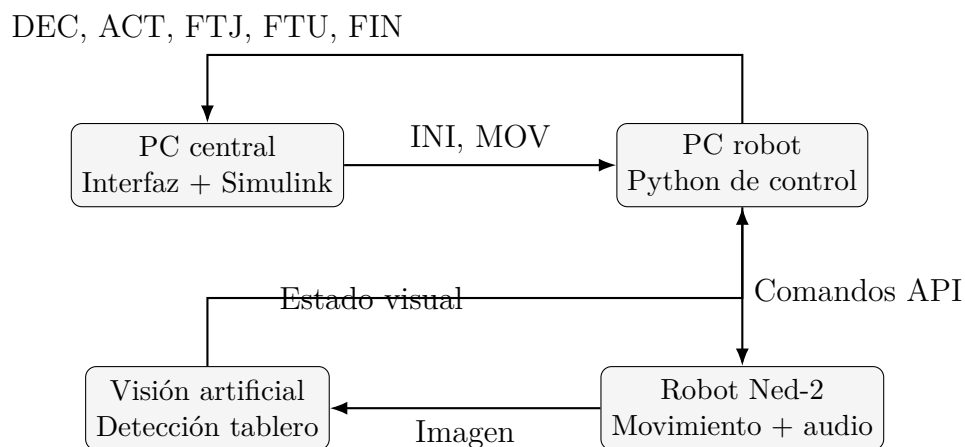


Figura 1: Gráfica de arquitectura funcional y flujo de información del sistema.

Subsistema	Tecnología principal	Función principal en el sistema
PC central	Matlab/Simulink	Supervisar estados, coordinar turnos y presentar estado de partida al usuario.
PC robot	Python + sockets	Ejecutar lógica de juego, control de movimiento y comunicación con el PC central.
Robot Ned-2	API pyniryo	Manipular piezas en tablero siguiendo restricciones cinemáticas de seguridad.
Visión artificial	OpenCV + cámara del robot	Detectar estado del tablero rival tras su turno y actualizar partida.

Tabla 2: Resumen de responsabilidades técnicas por subsistema.

3.2. Subsistemas software desarrollados

Los ficheros desarrollados para el PC del robot y el PC central permiten distinguir claramente las capas de responsabilidad del sistema. ComRobot.py concentra el bucle de partida, control de turnos, sockets y secuencias de movimiento; Ned2_ULL.py agrupa utilidades de configuración y persistencia de poses; y los dos bloques de Simulink materializan la comunicación bidireccional con el modelo del PC central.

Archivo	Tipo	Responsabilidad principal
ComRobot.py	Python	Bucle principal de partida, secuenciación de movimientos, gestión de turnos y mensajes de estado.
Ned2_ULL.py	Python	Utilidades de persistencia de poses, restauración de configuración y apoyo al robot.
SimulinkEmisor.m	MATLAB	Formateo de comandos INI/MOV para enviarlos al robot.
SimulinkReceptor.m	MATLAB	Parseo de mensajes recibidos y exposición de señales discretas en Simulink.

Tabla 3: Relación de componentes software desarrollados y su función técnica.

3.3. Características principales y decisiones de diseño por subsistema

3.3.1. PC central (Simulink + control de partida)

El PC central implementa el control supervisado en tiempo real solicitado en el enunciado. La máquina de estados en Simulink secuencia las fases de espera, orden de movimiento, confirmación de ejecución y evaluación de continuidad de partida. Esta representación permite validar transiciones de forma explícita y facilita la interpretación del estado por parte del operador.

La interfaz de comunicación se realiza mediante funciones MATLAB que convierten comandos y eventos entre señales de simulación y tramas de protocolo. Se adoptó una semántica de pulsos de un ciclo para eventos discretos (DEC, FTJ, FTU, FIN), evitando disparos múltiples y mejorando el acoplamiento temporal con el modelo de estados.

Los bloques MATLAB incluidos en el repositorio cubren de forma explícita la frontera de comunicación con el robot. SimulinkEmisor.m genera siempre un vector fijo de 32 bytes de tipo `uint8`, lo que simplifica su conexión con el bloque TCP/IP Send de Simulink y evita variabilidad de tamaño en tiempo de ejecución. El bloque soporta mensajes `INI MAN`, `INI AUT`, `INI COM` y `MOV origen destino`, incluyendo la traducción de identificadores de almacén 11–13 a las cadenas `alm1–alm3`. También contempla un modo de emisión de `MOV` sin argumentos para escenarios de sincronización o prueba.

Por su parte, SimulinkReceptor.m implementa un buffer persistente de recepción, acumula bytes procedentes del bloque TCP/IP Receive y procesa líneas completas terminadas en CR/LF. El diseño desacopla recepción y decodificación, permitiendo absorber fragmentación de red sin perder mensajes. Sobre ese buffer se reconocen los comandos `DEC`, `ACT`, `FTU`, `FTJ` y `FIN`. Para los eventos discretos se utilizan contadores persistentes que mantienen pulsos de 100 ms equivalentes en muestras, calculados a partir del periodo `Ts` del modelo. De esta forma, el resto de la máquina de estados recibe señales limpias y temporizadas.

En el caso del mensaje `ACT`, el bloque receptor transforma la cadena de nueve celdas del tablero en un vector numérico de longitud 9 apto para tratamiento en Simulink. Esta conversión permite que la lógica del PC central trabaje directamente con un estado discreto del tablero, sin necesidad de parseado textual adicional. El mensaje `FIN` se traduce además a una salida `RES` con codificación 1/2/3 para victoria, derrota o empate, lo que simplifica la integración con estados finales del modelo.

Bloque Simulink	Interfaz	Comportamiento implementado
SimulinkEmisor.m	Salida <code>uint8[1x32]</code>	Genera tramas de longitud fija para <code>INI</code> y <code>MOV</code> , con terminador de línea y compatibilidad directa con TCP/IP Send.
SimulinkReceptor.m	Entradas <code>data, count, Ts</code>	Acumula bytes en buffer persistente, procesa mensajes por línea y expone pulsos discretos y estado de tablero.
SimulinkReceptor.m	Salidas <code>DEC, FTU, FTJ, FIN, RES, tablePos</code>	Entrega eventos discretos temporizados y un vector de posiciones para la lógica de control central.

Tabla 4: Resumen funcional del código MATLAB/Simulink incluido en el PC central.

3.3.2. Subsistema robot (control de movimiento)

El control de movimiento del Ned-2 está implementado en ComRobot.py, donde se integra la recepción de comandos, la planificación de secuencias pick-and-place y la notificación de estado. El sistema soporta modos manual y automático sobre una arquitectura común de turnos, lo que permite conservar el mismo ciclo de ejecución independientemente de quién decide la jugada. Además de la conexión principal con el PC central, el código contempla una segunda conexión TCP/IP opcional para replicar mensajes de movimiento.

Se ha adoptado una estrategia de seguridad basada en desacoplo cinemático Z/X-Y, tal como exige el enunciado. El robot primero se posiciona en una pose de aproximación,

desciende verticalmente para recoger/dejar, y retorna al plano de aproximación antes de cualquier desplazamiento horizontal. Esta lógica se complementa con limitación de velocidad (valor 50) y retorno a la pose de visión general para el cierre de ciclo. El fin de turno rival se dispara mediante entrada digital DI1, tratada como evento de pulsador físico.

Elemento del Ned-2	Dato técnico	Uso en el proyecto
Grados de libertad	6	Permiten posicionar el efector final sobre el tablero y en la zona de reserva.
Sistema de visión	Cámara RGB integrada	Captura el tablero para detectar movimientos del rival.
Efector final	Pinza de dedos	Recogida y colocación de fichas.
Conectividad del robot	TCP/IP / rosbri- dge	Acceso mediante API pyniryo desde Python.
Límite de velocidad	50 en la configura- ción usada	Garantizar movimientos suaves y seguros.

Tabla 5: Resumen del hardware y configuración funcional del Ned-2 empleada en el proyecto.

3.3.3. Sistema de visión

El subsistema de visión utiliza la cámara integrada en el robot para actualizar el estado del tablero tras el turno rival. El pipeline de procesamiento incluye normalización geométrica del workspace, segmentación por color en HSV y asignación de detecciones a la rejilla 3x3. El resultado se publica como posiciones discretas en el protocolo de comunicación.

Una decisión de diseño especialmente relevante ha sido separar fuentes de verdad: el estado de fichas propias se mantiene por lógica interna, mientras que el estado rival se obtiene por visión. Esta separación reduce sensibilidad a errores de detección de las propias piezas y mejora la consistencia del ciclo de decisión automática.

Durante el desarrollo se incorporó una herramienta interactiva de calibración HSV mediante trackbars para facilitar ajustes según condiciones de iluminación. Esta funcionalidad fue especialmente importante en las pruebas iniciales, ya que el documento de requisitos no solo exige detectar la partida sino hacerlo de forma fiable y repetible en campo.

3.3.4. Comunicación TCP/IP

La comunicación remota se ha implementado sobre TCP/IP con mensajes ASCII terminados en fin de línea. Esta elección prioriza simplicidad de depuración y compatibilidad directa entre Python y Simulink, manteniendo suficiente expresividad para incluir acción solicitada, estado de proceso y estado de partida, tal como especifica el enunciado.

En la dirección PC central $\rightarrow$ robot se transmiten órdenes de inicialización y movimiento; en la dirección opuesta se reportan eventos de decisión, actualización visual y

cierre de turno. El receptor Simulink ya contempla mensaje FIN con resultado, pero su emisión en el bucle principal Python se considera tarea abierta de cierre funcional.

Estructura del mensaje	Contenido	Interpretación
IDN arg1 arg2 ... je	Identificador + argumentos	Formato genérico legible y compatible con parser por línea.
INI modo	Modo = MAN / AUT / COM	Inicio de partida en el modo seleccionado.
MOV origen destino	Origen y destino	Orden de recogida y colocación en modo manual.
DEC	Evento discreto	Confirmación de jugada decidida en modo automático.
ACT cadena9	Estado completo del tablero	Cadena de 9 celdas con símbolos _ / X / O.
FTJ / FTU	Evento de fin de turno	Cierre de turno del robot / del rival.
FIN res	VIC / DER / EMP	Soportado por receptor; emisión desde robot pendiente de cierre.

Tabla 6: Formato general de los mensajes intercambiados entre PC central y robot.

Mensaje	Origen	Momento de uso
INI modo	PC central	Justo antes de comenzar la partida.
DEC	Robot	Cuando la IA ha calculado su movimiento.
ACT cadena9	Robot	Tras capturar la imagen al terminar el turno rival.
FTJ	Robot	Al finalizar el movimiento físico propio.
FTU	Robot	Al cerrar el turno del rival humano.
FIN res	Robot	Definido en protocolo y receptor, pendiente de activación completa en bucle principal.

Tabla 7: Secuencia funcional de mensajes empleada durante una partida completa.

3.3.5. IA y decisión automática

La toma de decisión en modo automático implementada en la versión actual es heurística y ligera. En fase de colocación toma el centro (casilla 5) cuando está disponible en la primera jugada; en caso contrario selecciona una casilla libre aleatoria. En fase de movimiento selecciona de forma aleatoria una ficha propia y una casilla libre. Esta estrategia permite validar integración extremo a extremo con bajo coste computacional.

El enunciado fija además objetivos temporales concretos: 15 s máximo para recogida, 15 s para transporte y 10 s para decisión automática. En el código actual no existe instrumentación de tiempos ni verificación automática de dichas cotas; el comportamiento temporal se gobierna por esperas fijas en secuencias de movimiento y por coste mínimo de cálculo en la decisión heurística.

La mensajería de audio aparece como capacidad probada en scripts de apoyo y comentarios de operación, pero no forma parte del flujo activo de ComRobot.py en esta revisión.

Dirección	Mensaje	Información funcional transportada
PC central → robot	INI modo	Inicio de partida y selección de modo de operación (MAN/AUT).
PC central → robot	MOV origen destino	Orden de recogida y colocación de ficha en modo manual.
Robot → PC central	DEC	Confirmación de decisión tomada en modo automático.
Robot → PC central	ACT cadena9	Estado completo del tablero con codificación _ / X / O.
Robot → PC central	FTJ / FTU	Fin de turno del robot / fin de turno del rival.
Robot → PC central	FIN res	Resultado final de partida (VIC, DER o EMP), previsto en interfaz.

Tabla 8: Tabla de interfaz del protocolo de mensajería PC central-robot.

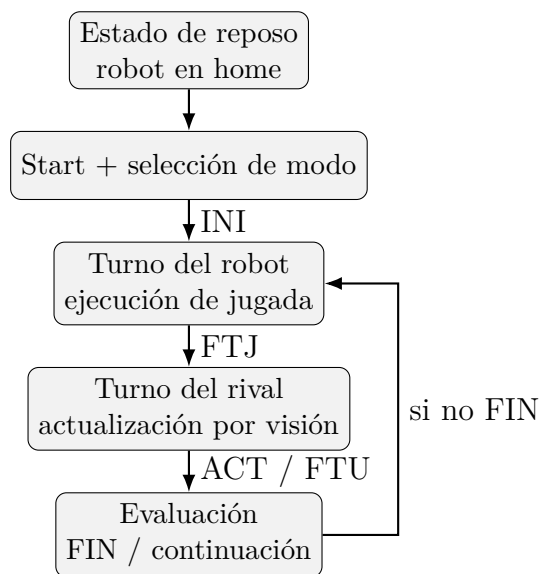


Figura 2: Gráfica de ciclo de turno en la máquina de estados del sistema.

3.4. Observaciones de consistencia detectadas en el código actual

Del análisis directo de los archivos Python/MATLAB se identifican los siguientes puntos de integración a cerrar en la versión final:

- En ComRobot.py existen funciones para verificar victoria y límite de turnos, pero actualmente no gobiernan el cierre del bucle principal de partida.
- El mensaje FIN está soportado por SimulinkReceptor.m, pero no se emite de forma sistemática desde el flujo principal del robot.
- SimulinkReceptor.m implementa mapeo de símbolos X/O a valores de tablero; conviene consolidar y documentar en un unico lugar la convención semántica de **rival** y **propio** para evitar ambigüedades.

Métrica temporal de enunciado	Objetivo	Respuesta de diseño actual
Proceso de recogida de pieza	≤ 15 s	Secuencias deterministas con esperas fijas; medición automática pendiente.
Proceso de transporte de pieza	≤ 15 s	Trayectorias desacopladas Z/X-Y; validación temporal en campo pendiente.
Toma de decisión en modo AUT	≤ 10 s	Heurística aleatoria de bajo coste; medición formal pendiente.

Tabla 9: Gráfica tabulada de objetivos temporales y solución técnica adoptada.

- No hay telemetría integrada para trazabilidad temporal automática de recogida, transporte y decisión.

4. Validación pormenorizada de requisitos

4.1. Procedimiento general de validación

Para cada requisito se seguirá el siguiente procedimiento:

1. Preparar el escenario de prueba (hardware, software y estado inicial).
2. Ejecutar el caso de validación.
3. Registrar el resultado observado (log, tiempo, estado o evento).
4. Comparar el resultado con el criterio de aceptación.
5. Marcar el estado: Cumple / No cumple / Pendiente.
6. Adjuntar evidencia con identificador unico.

Nomenclatura recomendada de evidencias:

- EV-RQ-XXXX-01_FOTO.jpg
- EV-RQ-XXXX-01_VIDEO.mp4
- EV-RQ-XXXX-01_LOG.txt
- EV-RQ-XXXX-01_CAPTURA.png

4.2. Estado observado en código actual (auditoría de implementación)

La tabla siguiente refleja el estado técnico observado directamente en el código activo del repositorio en esta revisión.

RqId	Estado actual	Fuente de evidencia	Evidencia técnica observada
000C-003	Implementado	Subsistema de visión del robot	Captura de imagen comprimida, extracción workspace y segmentación HSV.
000C-025	Implementado	ComRobot.py + SimulinkReceptor.m	Comunicación remota TCP/IP y parseo de eventos discretos.
000C-026	Implementado	ComRobot.py	Intercambio PC-robot con sockets, mensajes TX/RX y terminador \n.
000C-030	Implementado	Inicialización del control del robot	Límite de velocidad de brazo fijado a 50 en la configuración de arranque.
000C-034	Implementado	ComRobot.py	Fin de turno rival por lectura de entrada digital DI1.
000C-038	Implementado	ComRobot.py	Mensajes codificados como ASCII/UTF-8 con salto de línea final.
000C-009	Parcial	ComRobot.py	Modo manual activo con recepción MOV, sin validaciones de robustez completas.
000C-010	Parcial	ComRobot.py	Modo automático activo con heurística aleatoria; no minimax en versión actual.
000C-036	Pendiente	ComRobot.py	La función de victoria existe, pero no cierra el bucle principal de partida.
000C-037	Pendiente	ComRobot.py	MAX_TURNOS definido, pendiente de aplicación efectiva en flujo principal.

4.3. Requisitos pendientes por verificar (tabla preparada)

RqId	Nombre	Método	Procedimiento de validación (resumen)	Estado
000C-001	Sistema de juego 3 en raya	Test	Ejecutar partidas completas y verificar cumplimiento de reglas en todos los modos habilitados.	Pendiente
000C-005	Movimiento por turno	Test	Verificar que el robot solo se mueve en su turno y permanece en espera en turno rival.	Pendiente
000C-007	Piezas apiladas	Diseño + Test	Validar uso de pila mientras existan piezas y reubicación desde tablero al agotarse.	Pendiente
000C-008	Modos de juego	Diseño + Test	Comprobar disponibilidad y selección de modos en interfaz y máquina de estados.	Pendiente
000C-009	Modo manual	Test de sistema	Ejecutar batería de partidas completas para cerrar validación formal de robustez y errores de entrada.	En implementación
000C-010	Modo automático	Análisis + Test	Registrar decisiones automáticas y evaluar calidad de jugada de la heurística actual en campo.	En implementación
000C-011	Modo competición	Test	Ejecutar partida robot vs robot sin intervención manual.	Pendiente

RqId	Nombre	Método	Procedimiento de validación (resumen)	Estado
000C-012	Posición de inicio	Diseño + Test	Confirmar permanencia en home hasta Start.	Pendiente
000C-014	Selección jugador inicial	Diseño + Test	Verificar selector y correspondencia con primer turno.	Pendiente
000C-015	Cambio de modo restringido	Test	Probar cambio en partida y fuera de partida; validar bloqueo/permitido según estado.	Pendiente
000C-017	Desacoplo Z vs X-Y	Diseño + Test	Revisar trayectorias y observar secuencia vertical/horizontal en ejecución real.	Pendiente
000C-018	Recogida de piezas	Test	Comprobar aproximación X-Y y descenso vertical para agarre.	Pendiente
000C-019	Ascenso tras recogida	Test	Verificar ascenso vertical tras agarre antes de desplazamiento horizontal.	Pendiente
000C-020	Movimientos X-Y en plano de aproximación	Diseño + Test	Confirmar desplazamientos horizontales solo en plano de aproximación.	Pendiente
000C-022	Mensajes de audio	Test	Integrar audio en flujo principal y validar reproducción en estados clave: inicio, movimiento y fin de partida.	Pendiente
000C-023	Actualización por visión	Test	Contrastar tras cada turno el estado detectado con el tablero real y cerrar cobertura estadística.	En implementación
000C-024	Interfaz de usuario PC central	Diseño + Test	Verificar presencia y funcionamiento de elementos requeridos de interfaz.	Pendiente
000C-027	Estado del robot conocido	Análisis	Revisar trazas para comprobar disponibilidad continua del estado del robot.	Pendiente
000C-028	Formato PC→robot	Diseño + Test	Validar estructura y argumentos de mensajes de comando en trazas reales.	Pendiente
000C-029	Formato robot→PC	Diseño + Test	Validar estructura y argumentos de mensajes de estado/evento en trazas reales.	Pendiente
000C-031	Tiempo máximo de recogida	Test	Medir al menos 5 ejecuciones de recogida y verificar tiempo ≤ 15 s.	Pendiente
000C-032	Tiempo máximo de transporte	Test	Medir al menos 5 transportes completos y verificar tiempo ≤ 15 s.	Pendiente
000C-033	Tiempo máximo de decisión	Test	Medir tiempo de decisión en automático/competición y verificar ≤ 10 s.	Pendiente
000C-034	Fin de turno rival por botón físico	Test	Accionar botón físico y comprobar transición al siguiente estado en batería repetida.	En implementación

4.4. Plantilla de ficha de verificación individual

Para cada requisito pendiente, completar la siguiente ficha:

- RqId: [000C-XXX]
- Nombre: [Nombre del requisito]
- Fecha de prueba: [dd/mm/aaaa]
- Versión de software: [tag/commit]
- Configuración hardware: [resumen]
- Precondiciones: [estado inicial]
- Pasos ejecutados: [lista de pasos]
- Resultado observado: [descripción objetiva]
- Criterio de aceptación: [criterio]
- Decisión: [Cumple / No cumple]
- Evidencias asociadas: [lista de archivos]
- Incidencias detectadas: [si/no + detalle]

5. Guía rápida para la puesta en marcha

5.1. Prerrequisitos

Hardware mínimo:

- Robot Ned-2 calibrado.
- Cámara del robot operativa.
- PC central y PC de control del robot conectados por red.
- Tablero y piezas en posición inicial.

Software mínimo:

- Entorno Python con dependencias del proyecto.
- Modelo Simulink con bloques de emisión/recepción configurados.
- Scripts de control de robot y visión disponibles.

5.2. Secuencia de arranque recomendada

1. Verificar conexión de red entre PC central y PC del robot.
2. Encender y calibrar el Ned-2.
3. Cargar configuración de poses y parámetros de visión.
4. Iniciar servicios/scripts de control en el PC del robot.
5. Abrir el modelo de control en PC central (Simulink).
6. Configurar IP/puertos y arrancar la comunicación.
7. Confirmar estado home y seleccionar modo de juego.
8. Pulsar Start para iniciar partida.

5.3. Comprobaciones de arranque (smoke test)

- Prueba de conectividad entre equipos.
- Envío y recepción de mensaje de prueba TCP/IP.
- Verificación de un movimiento simple del robot.
- Verificación de captura de imagen y detección básica.

5.4. Checklist de configuración inicial

- Red operativa y parámetros IP correctos.
- Robot en home antes de Start.
- Piezas en posición de inicio (almacenes).
- Modo de juego correcto seleccionado.
- Registro de logs activado para evidencias.

5.5. Incidencias frecuentes y acción rápida

Síntoma	Posible causa	Acción recomendada
No hay conexión PC-robot	IP/puerto incorrecto o firewall	Revisar configuración de red y reglas de firewall
Visión detecta mal las piezas	Umbrales descalibrados o iluminación variable	Recalibrar HSV y estabilizar iluminación
Movimiento incompleto	Pose mal definida o timeout	Revisar poses, límites de velocidad y logs

Síntoma	Posible causa	Acción recomendada
Start no inicia	Robot no está en home	Llevar robot a home y repetir inicio

6. Anexos de evidencias

6.1. Índice de evidencias (plantilla)

ID evidencia	Requisito	Tipo	Descripción	Estado
EV-RQ-000C-001-01	[000C-001]	Video	Partida completa de validación	Pendiente
EV-RQ-000C-023-01	[000C-023]	Captura	Estado detectado por visión	Pendiente
EV-RQ-000C-033-01	[000C-033]	Log	Tiempos de decisión en modo automático	Pendiente

6.2. Anexo de trazas de comunicación (pendiente)

- Incluir logs crudos PC central → robot.
- Incluir logs crudos robot → PC central.
- Incluir resumen de secuencias por modo de juego.

6.3. Anexo fotográfico/video (pendiente)

- Evidencias de montaje.
- Evidencias de ejecución por requisito.
- Evidencias de interfaz y estados.

7. Cierre (plantilla)

En la versión final de este documento se incluirán:

- Resultado final por requisito (Cumple / No cumple).
- Justificación de desviaciones, si las hubiera.
- Riesgos residuales y recomendaciones de mejora.
- Conclusiones técnicas definitivas.